

Lecture 7: Media Device SW Platform – Middleware (AL)

In this lecture, we move deeper and up the software stack of a media device and focus on the middleware layer. In previous lectures, we discussed host software as a whole, detailing HAL and PAL layers. We now open up the Middleware – Application Layer and finish up the media device software story.

The main focus will be the Digital TV middleware block, because this is where most of the broadcast-specific logic lives. Digital television is not only about playing audio and video. A complete broadcast TV system must scan services, parse transport streams, build service lists, extract program information, manage playback, support subtitles and teletext, and often interact with recording, reminders and application engines. These functions are too specialized and too complex to be handled only by a simple application layer.

Before going deeper into the middleware itself, it is useful to remember that we have been working with two different flavours of the stack: one based on Linux and one based on Android. Up to the platform level, these two stacks can look relatively similar, because both need access to tuners, demultiplexers, decoders, audio and video output, storage and network functionality. However, once we move into the application runtime and middleware integration, the architecture becomes quite different. In this lecture, the focus is on the Android version of the stack.

AL in the Android Perspective

Android applications run on the Android Runtime, or ART. ART replaced the older Dalvik virtual machine and is responsible for executing application code packaged as APK files. This runtime forms the execution environment for Android applications, including applications used on Android TV devices.

Above the runtime, Android provides the Android application framework. This framework contains the standard services used by Android applications. The Activity Manager handles application lifecycle and transitions between application components. Content providers manage structured data that can be shared between components. There are also managers for resources, packages, notifications and other system-level functions that application developers use regularly.

More recently, Android development has also become strongly connected with Jetpack, a collection of libraries and tools that help developers build applications in a more structured and consistent way. Modern Android applications commonly rely on Jetpack components, and user interfaces are often developed using Jetpack Compose with Kotlin.

From a media perspective, Android also provides media player libraries and playback-related APIs. If we were building a general media application, this would already give us a strong toolbox: application framework services, UI components, media playback APIs and a managed runtime.

However, none of this is specific to digital television. These are general Android facilities. A normal Android media application can use them, but a DTV application needs much more: it must integrate with live TV concepts, service lists, broadcast scanning, service identifiers, transport stream processing and Android's TV-specific framework. This is where the middleware and the Android TV Input Framework become central.

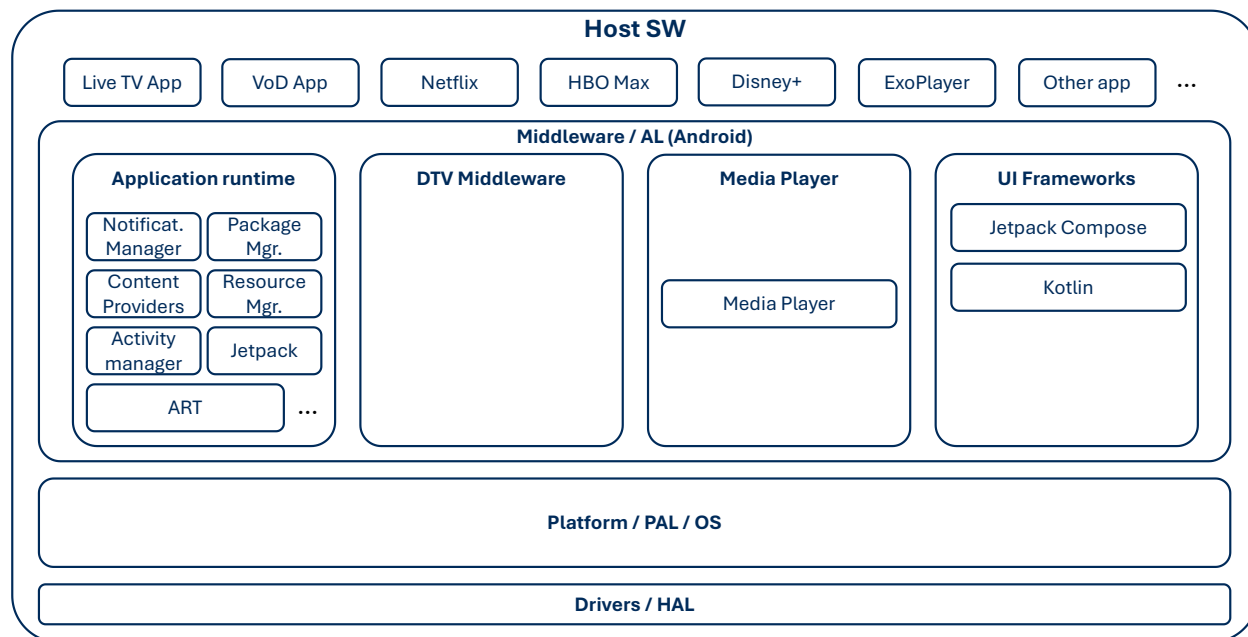


Figure 1. Android version of the middleware (AL)

The TV Input Framework as Android's TV Abstraction

Under Android, the key abstraction around TV functionality is the TV Input Framework, or TIF. This framework is extremely important for Android TV development, and it will also be important for the project work. The main idea behind TIF is that Android wants all TV services on a device to appear in a unified way.

In older or more vertically integrated TV systems, one TV application could be tightly connected to one private middleware implementation and one platform stack underneath. This approach creates a device flavour dedicated to a specific telecom operator, broadcast operator or content provider. Operators may like this model because they often want only their own content to be visible and controlled through their own application.

Android has a different platform goal. It wants multiple content sources to coexist on the same device. A single Android TV device may contain broadcast services, IPTV services, OTT services and possibly other video sources. Android therefore needs a framework

through which different service providers can expose their content without each one replacing the complete TV user experience.

With the TV Input Framework, the Live TV application can present a unified user experience: a channel list, service switching, service information and playback. The actual services, however, may come from different TV inputs. One TV input may represent broadcast. Another may represent IPTV. Another may represent OTT. Additional inputs can be added for other sources.

This is the design pattern: every TV input contributes its own services, and every TV input reacts to commands from the TV Input Manager. When the user selects a service in the Live TV application, the TV Input Manager determines which TV input owns that service. It then calls the appropriate callback on that TV input.

One of the most important callbacks is onTune. The word “tune” must be understood carefully here. At the tuner or frontend level, tuning means selecting a physical frequency. At the TIF level, however, tuning is a higher-level operation. It means something closer to “zap to this service” or “activate this selected service.” The selected service may come from a broadcast frequency, from an IPTV stream, from an OTT URL or from another source. Therefore, onTune is not necessarily an RF tuning operation. It is a middleware-level or service-level activation request.

This distinction is important for Project 2. The project requires a design compatible with the TV Input Framework. The system must define the required TV inputs, implement them and make sure they can properly serve the Live TV application above.

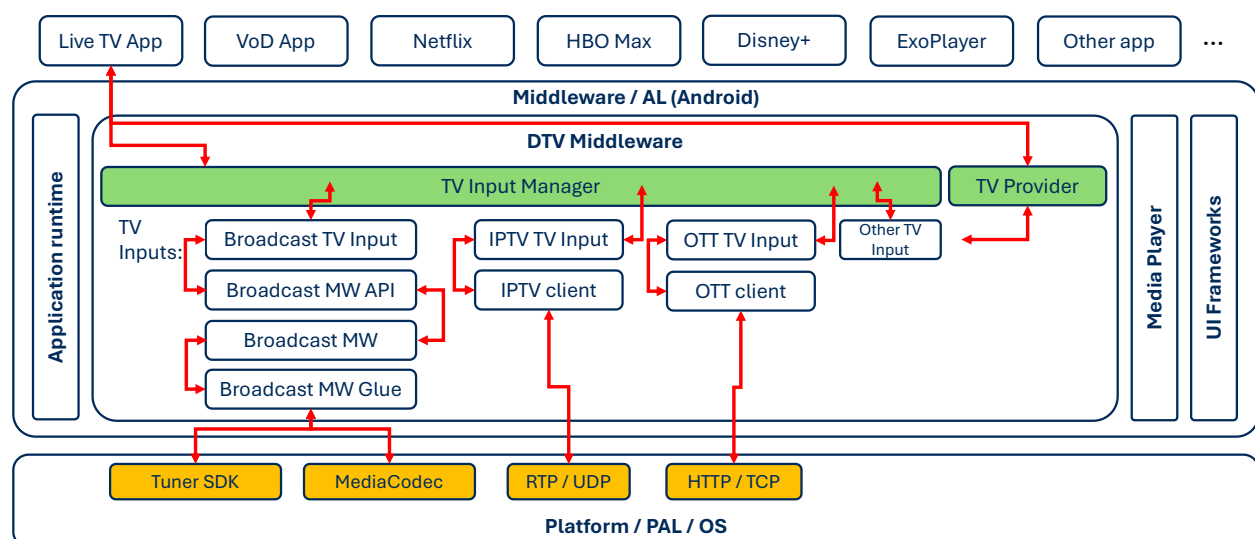


Figure 2. TV Input Framework as DTV Middleware in Android

Broadcast TV Input and Comedia Middleware

In the broadcast case, the Broadcast TV Input communicates with a DTV middleware service. In the project environment, that middleware service is Comedia, a DTV middleware solution from iWedia.

Comedia helps implement most of the broadcast-specific logic required by the TV input. It exists mostly as native middleware. It exposes a Java-facing API, but this API binds to native code through JNI. The native code then talks to the platform underneath.

This gives us several layers. At the Android application side, the Broadcast TV Input calls into the Java-facing Comedia service. From there, calls enter the native middleware through a middleware abstraction layer. The middleware core performs the broadcast-specific logic. Below the core, a glue layer connects the middleware to the actual platform APIs.

This glue layer is closely related to the wrapper discussion from the previous lecture. Its purpose is to allow the middleware to remain as hardware-independent and platform-independent as possible. Comedia is valuable because it contains complex DTV logic. Ideally, the same middleware core should run on different targets: Linux, Android, Broadcom-based systems, other SoCs and different platform APIs. To make that possible, the middleware cannot be tightly bound to one specific interface such as Nexus, Android Tuner SDK or any single vendor SDK.

Instead, Comedia defines its own internal platform-facing API. This can be understood as a middleware-defined platform abstraction layer. It is not the official platform API of the operating system, but an abstraction defined by the middleware itself. The middleware core calls this glue API, and different glue implementations resolve the same calls for different targets.

There may be one glue implementation for Android, another for Linux, another for Android with Broadcom, another for Linux with Broadcom, and so on. The glue changes, but the core DTV middleware remains mostly unchanged. This is the main reason why the glue layer exists.

Underneath the glue layer, there is still a real platform implementation. In one setup, the middleware may communicate directly with Nexus. In a cleaner Android-aligned architecture, it should ideally go through Android's Tuner SDK and Tuner HAL. In practice, real products often contain a mixture of older and newer approaches.

The Android stack can be quite thick. A user-level operation may pass through the Tuner SDK, Tuner Native API, Tuner Resource Manager, Tuner HAL and finally the vendor implementation such as Nexus. This may look heavy, but Android introduced framework and HAL boundaries for a reason. They make system updates and platform separation easier. If the upper Android framework is tightly coupled to vendor code, every Android

update becomes painful. If the vendor layer implements expected abstractions, the upper system can evolve more independently.

For that reason, the cleaner Android architecture is for broadcast middleware to go through the Tuner framework and Tuner SDK rather than bypassing Android's abstractions completely.

IPTV and OTT Compared with Broadcast Middleware

The IPTV and OTT cases are usually simpler than broadcast from the middleware point of view. IPTV and OTT clients still need to handle metadata and streams, but they do not normally require the same heavy broadcast-specific machinery.

In managed IPTV, the server is usually inside a controlled network. Because the network is managed, the system may use RTP, UDP or custom ports to optimize delivery and bandwidth. This differs from open Internet delivery, where assumptions about the network cannot be as strong.

In OTT systems, adaptive streaming over HTTP and TCP is commonly used because the content must work across the open Internet and across many device types. The client obtains metadata from servers and plays media streams using standard playback mechanisms. In Android, much of the actual OTT playback can be handled using existing Android media playback APIs.

Broadcast is more complex because it must support standards such as DVB-T or ATSC, parse many PSI/SI tables, manage transport streams, route PIDs, interact with demux hardware, configure accelerated decoder paths and maintain service databases. This is why the broadcast middleware block is significantly heavier than the IPTV or OTT client-side blocks.

The Role of the Android TV Provider

Another important component in the Android TV architecture is the TV Provider. Android cares about the overall user experience. It wants content from different sources to appear consistently in search, menus, recommendations, channel lists and other system surfaces. The TV Provider is the Android component that stores information about TV channels, programs and related metadata in a database-like form.

If a TV input discovers services, for example through a broadcast scan, it should not keep that information only inside its private middleware database. It also needs to publish relevant information into the TV Provider. This allows Android to include those channels and programs in the broader system experience.

In the broadcast case, the middleware performs a service scan and builds its own service database. The Broadcast TV Input must retrieve that information from the middleware and insert or update it in the TV Provider. This connection is essential for proper Android integration. Without it, the middleware might know which services exist, but Android would

not be able to present them consistently through the Live TV application and other system-level surfaces.

The overall Android integration task can therefore be summarized as follows: implement the APIs required by the TV Input Framework, use the middleware service to satisfy those API calls, understand the flow down to the platform, and keep the TV Provider informed about the services and programs discovered by the middleware.

In the concrete project case, the stack becomes Comedia API, Comedia middleware and Comedia CHAL, where CHAL is the Comedia Hardware Abstraction Layer, or the glue layer.

Major Layers in the Comedia-Based Middleware Stack

In a Comedia-based broadcast middleware example, we can think of three major layers.

At the top, there is the API side of the middleware service. In Comedia, this upper API side has two parts: the Java-facing Comedia Service and the native middleware abstraction layer, often called MAL. These APIs expose functional groups toward the TV input and, eventually, toward the application layer.

Below the API side is the middleware core. This is where the main DTV functionality is implemented. It contains the internal logic for scanning, service management, playback routing, table parsing, EPG handling, recording, reminders and other broadcast-specific functions.

Below the middleware core is CHAL, the Comedia Hardware Abstraction Layer. CHAL is the glue layer that connects middleware functionality to the real platform implementation. Under CHAL, the system may use Nexus directly, Android Tuner SDK and Tuner HAL, or another platform-specific implementation.

The important idea is that the upper APIs expose functional groups, while the deeper middleware components perform the actual broadcast work. Android sees a centralized and consistent TV experience, while the middleware hides much of the complexity underneath.

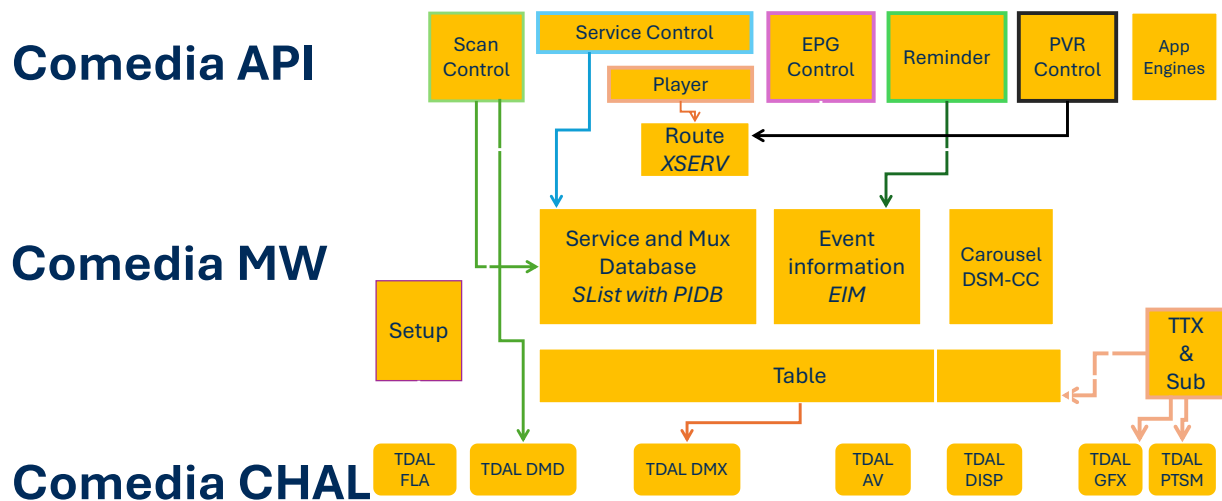


Figure 3. Internal structure of Comedia DTV Middleware

Main Functional Groups of DTV Middleware

One of the first major functional groups is Scan Control. Scan Control handles installation. Anything related to service scanning belongs here: starting a scan, reacting to a found frequency, reacting to a found service and gradually building the service database. It drives the process of discovering what exists in the broadcast network.

Service Control is the central point for controlling services and playback selection. When the user switches service, the request goes through Service Control. When the application needs information about a service, it also goes through Service Control. This makes Service Control one of the most important API groups, because it connects the user's intention with the internal broadcast model.

The Player abstraction handles actual playback operations. It starts playback, stops playback and coordinates playback state. It is naturally coupled with Service Control, because selecting a service and playing that service are closely related operations.

EPG Control handles schedules and program information. In broadcast TV, this information is carried in Event Information Tables, or EIT. These tables provide the electronic program guide: what is currently being broadcast, what comes next and what will be shown over the following days. EPG Control exposes this information upward.

Reminder functionality handles scheduled reminders and scheduled recordings. This is important because users may want to be notified before a program starts, or they may want to schedule a future recording based on EPG data.

PVR Control handles recording functionality and access to stored recordings. PVR stands for Personal Video Recorder. It includes use cases such as recording a live service, listing recordings and playing recordings later.

There are also application engines such as MHEG and HbbTV. These technologies allow broadcast services to provide interactive or enriched content. For example, on a service that supports HbbTV, a red-button prompt may appear. When the user presses the red button on the remote, the TV can open an interactive portal associated with that service. Historically, this was a way to create a hybrid experience where broadcast remained central and internet access was an additional optional channel. The user could receive enriched service information, portals, catch-up content or interactive pages related to the broadcast service. However, because general Internet applications became dominant, broadcast-centered application-engine technologies did not become as universal as they might have.

Internal Components of the Broadcast Middleware Core

Inside the middleware core, one of the most important components is the Service and Mux Database. It may also be represented as SList with PIDB, where PIDB stands for Program Information Database. This is a large persistent database that stores information about multiplexes, services, PIDs and related metadata. It is the central storage point for what the middleware has discovered about the broadcast network.

Scan Control fills this database. To do that, it must use frontend-related functionality through the glue layer. In Comedia CHAL terminology, this may involve modules such as TDAL DMD, corresponding to demodulator or frontend control. Underneath, this is eventually resolved to something like Nexus frontend control or to an equivalent platform API.

To populate the database, the middleware must parse many broadcast tables. This is the responsibility of the Table module. The Table module parses PSI/SI tables and provides the results to other middleware components. To do that, it needs access to the demux so that it can obtain table sections from the transport stream. Through CHAL, this goes through modules such as TDAL DMX, which underneath map to platform abstractions such as Nexus ParserBand and PidChannel, or to Android Tuner filter abstractions.

The Player interacts with a routing component called Xserv. Although Xserv may look like a small block in a diagram, it is extremely important because it manages the current signal path and playback configuration. When the Player is told to start, Xserv must orchestrate the complete system. It must tune or connect to the right input source, configure the demux, set the required PIDs, activate audio and video decoders, configure the display layer and maintain playback state.

In other words, Xserv is where abstract service selection becomes a concrete signal path.

There is also a separate Event Information database, often handled by the Event Information Manager. This is where schedule data is stored. Reminder functionality uses this data to schedule reminders or recordings.

Application engines use DSM-CC carousel functionality. DSM-CC allows files and data objects to be delivered inside the broadcast stream. For example, an HbbTV or MHEG application may obtain icons, images or other resources from the stream itself, even without Internet connectivity. This is a specialized part of the stack, but it explains how broadcast can carry more than audio, video and classical PSI/SI metadata.

Teletext and subtitle parsing are also present as specialized modules. They correspond to selective PES parsing. These modules obtain information from the table system, parse the corresponding PES or subtitle data and produce content that is eventually rendered through graphics or presentation modules. Subtitles also require timing synchronization, often through a PTS management module.

Finally, there is a setup or configuration module. This module persists user and system configuration into flash storage. It may store the service list, mux list, last watched service, module settings and other persistent state. Without such persistence, the device would need to rediscover or reconfigure everything after each restart.

When we look inside DTV middleware, we therefore see a heavy but logical system. It stores service information, parses tables, controls scanning, manages playback, handles EPG and reminders, supports recording, parses subtitles and teletext, and connects all of that to the platform through a glue layer.

Comedia as an Android Service

Specific to Comedia, the middleware is realized as a separate Android service. The Broadcast TV Input implementation accesses Comedia APIs by binding to this service using Android's Binder mechanism.

This means that the TV input does not directly contain the whole middleware. Instead, it communicates with a middleware service. The service boundary is important because it separates the Android TV input implementation from the native DTV middleware. The TV input issues high-level requests, while Comedia performs the actual broadcast-specific work and reports results back through callbacks and events.

Channel Scanning Flow

A useful way to understand the middleware is to examine a concrete channel scanning sequence. The sequence described here is simplified, but it is based on a real working implementation and is therefore useful for understanding how the layers cooperate.

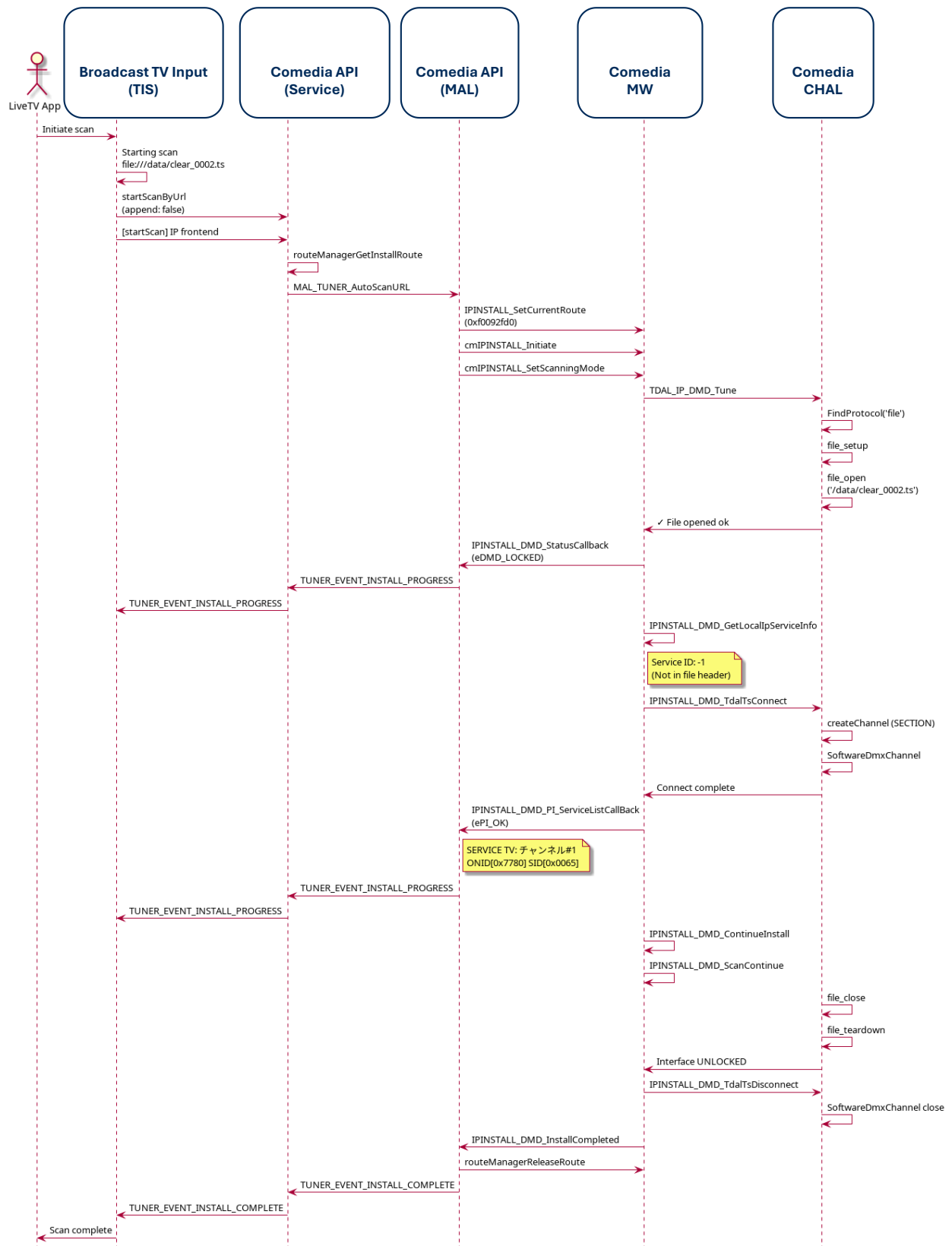


Figure 4. Channel scanning UML sequence

The example comes from the codebase used in Project 2. Since the project runs in a simulated Cuttlefish environment, there is no actual broadcast tuner. Instead, a transport stream file is used to simulate the tuner. For this reason, the flow is based on a Comedia `scanByUrl` call rather than a real broadcast frequency scan. Conceptually, however, the process is the same: the middleware receives transport stream data, parses broadcast tables, discovers services and builds a service database.

The flow starts in the Live TV application, where the user initiates a scan. This usually happens when the device does not yet know which services are available, for example after first installation. The system must discover what can be watched.

The request goes to the TV Input Service. In this example, the TV Input Service is not scanning a real terrestrial frequency. Instead, it scans a local transport stream file. This file simulates what would normally arrive from a broadcast tuner. MPEG-TS packets are read from the file and fed into the system as if they were arriving from a live signal.

The TV Input Service calls `startScanByUrl` toward the Comedia service. The URL points to the local TS file. The parameter `append: false` means that newly found services should not be added to an existing service list. Instead, the old list should be cleared and a fresh service list should be created.

From the Comedia service, the request enters the native middleware through the MAL layer. The middleware first obtains an install route. A route is the middleware abstraction of the processing path needed for a specific use case. For scanning, the route is simpler than for live playback. It mostly needs an input source and a demux path, because the goal is to obtain and parse tables. If this were live playback, the route would also include audio and video decoders, display output and synchronization paths.

The middleware then starts the installation procedure. It sets the current install route, initiates the installation module and sets the scanning mode. This matters because scanning can mean different things. It may mean an automatic scan over a range of frequencies, a manual scan on one frequency, a network scan that follows NIT information, or, as in this case, an IP- or URL-based scan.

The middleware then calls down into CHAL through a function such as `TDAL_IP_DMD_Tune`. In a normal tuner case, this would tune the frontend to a frequency. In the file-based case, it opens the URL, detects that the protocol is `file`, sets up file reading and opens the local TS file.

Once the file is opened, reading starts. A thread may read the file packet by packet, or in blocks of packets, and push that data toward the demux path. Conceptually, this replaces the live tuner signal with a software-fed transport stream.

When the file is successfully opened, the lower layer reports a lock-like event back upward. The name may look like a demodulator lock event, even though no real RF tuning happened. This is intentional. The middleware wants to reuse the same abstraction.

Whether the source is a tuner or a file, the upper middleware receives a similar notification that the input is ready.

After the input becomes ready, table parsing begins. The middleware configures the demux and table-processing modules to obtain the tables required for installation. For a basic service scan, this usually includes PAT, PMT and SDT. PAT tells the middleware which services exist and where their PMTs are. PMT tells the middleware which elementary streams belong to a service. SDT provides service descriptions and names.

When table data is parsed, callbacks propagate upward as installation progress events. The middleware reports that something has been found, and the TV Input Service can forward that progress toward the Live TV application.

In the middle of the scanning sequence, the middleware requests local service information. It connects table parsing to the transport stream and creates the required section channels. In this file-based case, a software demux channel is involved. Once the service list is parsed, the middleware reports a service-list callback. In the example from the project environment, one service is found, with identifiers such as ONID and SID. The human-readable service name is shown in Japanese because the TS file comes from such a stream sample.

The discovered service is then written into the middleware service database. After this step, the service becomes available for selection and later playback.

The scan continues because the middleware must check whether more services are present. In this example, the file contains only one service, so after completing the scan content, the system starts cleanup.

Cleanup is important. The file is closed, the file protocol instance is torn down and the input becomes unlocked. The middleware disconnects the modules used for scanning, especially the input and demux path. Software demux channels are closed, and the install route is released so the same resources can be used later for playback or another operation.

Finally, the middleware sends the installation-completed notification upward. This becomes a `TUNER_EVENT_INSTALL_COMPLETE` event toward the Comedia service, then toward the TV Input Service and finally toward the Live TV application.

In simple terms, the user starts a scan, the TV input asks Comedia to scan a URL, Comedia creates an install route, opens the TS file as an input source, feeds packets to the demux, parses broadcast tables, discovers services, stores them in the service database, cleans up the route and reports that the scan is complete.

This sequence is a good example of why middleware is complex. From the user's perspective, this is only "scan channels." Inside the stack, however, it includes routing, input setup, protocol detection, file reading, demux configuration, table parsing, service database updates, progress callbacks and cleanup.

Broadcast Zapping Flow

After scanning, at least one service exists in the service list. The user can now select a channel and start watching it. This operation is usually called a zap. In Android terminology, it appears as onTune, but conceptually it is not only tuning to a frequency. It means activating the selected service.

The sequence again starts in the Live TV application. The user selects channel 1 from the list, and the request goes through the TV Input Framework toward the TV Input Service.

Before playback can actually start, the TV input must receive a video surface. The middleware itself does not own the Android UI and does not automatically know where decoded video should be displayed. The graphical application provides a surface, and this surface is passed to the TV input through onSetSurface. Later, that surface must be connected to the decoder or rendering pipeline so that video has somewhere to appear.

In parallel, the TV input also receives onSetStreamVolume. In the example, the volume is set to zero at the beginning. This is a common practical trick. During channel switching, the output may be muted until the system is sure that the correct service is active and stable.

The key call is then onTune. The Live TV application selected a channel, but the reliable broadcast identifier is not only a channel name. The reliable identifier is the service triplet: ONID, TSID and SID. ONID is the Original Network ID, TSID is the Transport Stream ID and SID is the Service ID. Together, these three values uniquely identify the service.

The TV input passes this selected triplet toward Comedia using a call such as setCurrentServiceByTriplet. Before the service can be started, the middleware obtains a live route. Compared to the install route used during scanning, the live route is more complete. It needs the input source, demux, audio and video decoders, synchronization and display output.

The request then enters the Comedia layers: the Java-facing service, the native MAL layer and finally the Comedia core. Starting playback is asynchronous. A call such as MAL_SZ_StartTripletService does not block until the whole channel is visible. Instead, it posts a message to the main zapper task, which continues the zap operation in its own thread.

This is very important for Project 2. Many media operations are asynchronous. A function call may return quickly, while the real result arrives later through a callback. A zap may take two or three seconds, so the application must wait for a service-started event rather than assuming that playback has started immediately.

In this example, the selected service is again backed by the same local TS file used during scan. The middleware obtains tuner or input control, detects that the protocol is file, opens the file and starts reading it. A read thread pushes TS packets into the system, just as a tuner would normally push live transport packets.

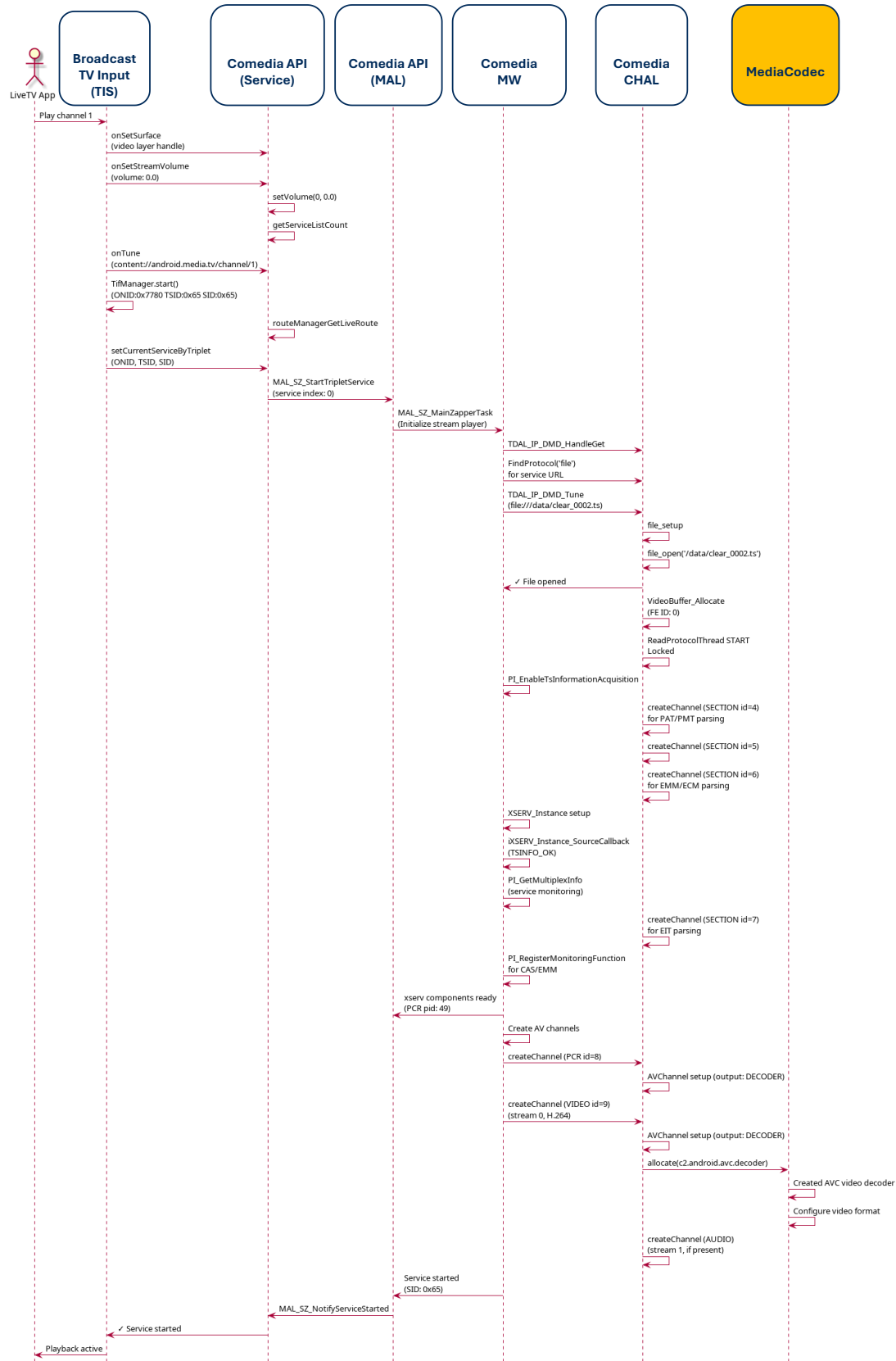


Figure 5. Broadcast zapping UML call sequence

The middleware then enables transport stream information acquisition. Even during zap, the system still needs PAT and PMT information in order to discover the actual audio, video and PCR PIDs for the selected service. If the service were scrambled, the middleware could also open ECM and EMM parsing paths for conditional access, but that can be ignored in the simplified project explanation.

At this point, Xserv becomes central. Xserv is the routing and playback orchestration component. It creates the playback instance, waits until transport-stream information is available, monitors the service and coordinates the remaining setup.

Once the needed tables are parsed, the middleware knows the components of the service. It reports that the components are ready. It can then open the A/V channels: one for video, one for audio and one timing path based on PCR. PCR is the clock reference used to synchronize audio and video so that frames and samples are presented at the correct time.

The video channel is connected to Android's MediaCodec. In the example, the codec is allocated for AVC, meaning H.264 video. On a non-Android system, or in a direct vendor implementation, this could instead map to a vendor decoder such as a Nexus video decoder. In the Android environment, MediaCodec is the abstraction used for decoding.

Audio is opened through the corresponding audio path, typically AudioTrack or a similar Android audio playback abstraction.

When the input, demux, timing reference, video decoder and audio decoder are all configured and started, Comedia reports that the service has started. That notification travels back upward through MAL, Comedia Service, the TV Input Service and finally to the Live TV application.

From the user's perspective, a channel was selected and playback became active. Internally, however, the stack opened a live route, identified the service by triplet, started an asynchronous zapper task, opened the transport source, parsed tables, discovered PIDs, configured monitoring, opened A/V channels, allocated MediaCodec and started playback.

Middleware APIs to Study

As part of this lecture, students should analyze the following APIs: TV Input Framework with TV Input Service and Comedia MW. Understanding these middleware APIs is important for Project 2 because the implementation will depend on calling middleware functions correctly, reacting to asynchronous events and mapping Android TV Input Framework operations to the underlying Comedia service.

The most important point is not only to recognize individual function names, but to understand the role of each API group in the complete flow. Scan-related APIs support installation and service discovery. Service-control APIs support service selection and zapping. Playback-related APIs control the actual media pipeline. The project

implementation should connect these API groups to the Android TV input callbacks in a consistent way.

Project 2: Building a TV App in a Simulated Android Environment

Project 2 applies the lecture concepts by requiring students to build an actual TV application in a simulated Android environment.

In Project 1, the focus was on different use cases, signal paths, dataflows and control sequences. Project 2 is more concrete. The class now works on one specified use case and implements it in software under Android.

The goal is to build a TV application running in the Cuttlefish Android emulator. Conceptually, the same application could also run on an Android-based set-top box.

The application must support service zapping using a service list. Zapping means switching from one service to another. The user should be able to move through the service list, select a service and start playback. The application should also display basic service information.

The application must support two types of services. The first type is broadcast service. In the project environment, broadcast is simulated through a transport stream file. Instead of using a real tuner signal, the system feeds the middleware with a TS file, but the logic should still represent the broadcast pipeline.

The second type is OTT service. For OTT playback, students will mostly rely on standard Android playback from a URL. Android already supports this well, so there is no need to implement adaptive streaming itself. The more important part is communication with the OTT middleware side: retrieving metadata from a server, parsing JSON or XML and building the OTT service list.

The application must also support first-time installation of services from all available sources. This means it must perform the scan or discovery process, populate internal service databases and update the Android TV Provider where needed.

All of this must be integrated into the Android Live TV application model. Therefore, the implementation must respect Android TV Input Framework design patterns.

Technically, the solution must create two TV Inputs: one for broadcast and one for OTT. Each TV Input must follow the TIF model. The Broadcast TV Input should use the Comedia DTV Middleware service for scanning, service discovery, service selection and playback control. The OTT TV Input will be more custom. It will use Android media playback for the actual stream, while metadata retrieval and parsing will be implemented inside the TV Input logic.

For metadata, both worlds need at least service names. Broadcast service names come through Comedia middleware mechanisms. OTT service names and optional descriptions come from the OTT server.

The development environment is Android Studio with Android 16 running in Cuttlefish. The final demo must show that the application works in that environment.

Project Deliverables

Project 2 is divided into design and implementation deliverables.

For Part 1, students deliver the architecture and design. The first required item is an architectural block diagram showing the software blocks across all layers: HAL, PAL and AL. These layers should be populated with the blocks used in the proposed solution.

The second required item is a set of UML sequence diagrams. Unlike Project 1, where the emphasis was mainly on commands and dataflow, Project 2 focuses more strongly on API calls. The diagrams should resemble the examples discussed in the lecture, but they must be adapted to the student's own solution.

For HAL and platform layers, demonstration-level detail is acceptable. Students do not need to document every low-level call. However, for the application and middleware parts, the sequence diagrams should be much closer to implementation level, because those are the parts that will actually be coded.

At minimum, the design must include sequence diagrams for service installation, broadcast zapping and OTT zapping. Students should also add one more diagram of their choice, for example a diagram related to metadata handling.

For Part 2, students deliver the implementation. This includes the design specification updated with requirements, the architecture block diagram and the UML sequence diagrams. It also includes source code aligned with that design, a working APK and a live demo.

The idea is straightforward: in Part 1, students design the architecture. In Part 2, they implement it. The architecture should not be decorative. It should guide the code.

The first deadline is for the architecture and design part. After that, the work moves into the implementation phase, where the design is brought to life in the lab environment.

Closing Perspective

Middleware is often invisible to the user, but it is one of the most important parts of a digital TV device. When the user scans channels or switches from one service to another, the visible interaction looks simple. Internally, however, the system performs route creation, input configuration, demux setup, table parsing, service database updates, PID routing, decoder allocation, synchronization and callback handling.

Android adds another layer of structure through the TV Input Framework and TV Provider. These components make it possible for different TV sources to coexist inside a unified system experience. Broadcast, IPTV and OTT can be integrated through separate TV inputs, while Android presents them to the user through consistent UI concepts.

Project 2 brings these ideas together. Students must design and implement a TV application that respects Android TV architecture, uses Comedia for broadcast middleware functionality, handles OTT service metadata and playback, updates the relevant service information and demonstrates zapping in a simulated Android environment.

This is the point where the architectural story becomes practical software engineering. Understanding the middleware layer is therefore not only theoretical; it directly shapes how the TV application must be designed and implemented.